

CLASH OF CLANS

S31371 Danylo Antypenko

S29991 Odul Lorelei Kafadar

S29778 Konrad Markiewicz

S30917 Danila Novik

User Requirements	2
Use-Case Diagram	4
Class diagram - Analytical	5
Description	7
Class diagram - Design	8
Design Decision	10
Use Case and Activity Diagram	13
1:Building a Building	14
2:Train and Chill Units	16
3:Initiate an Attack and Apply Battle Result	18
4:Join Clan With Ban	19
State Diagram	22
Building State Diagram	22
Description	22
Join Clan Player	23
Description	23
Changes to the Class Diagram After Dynamic Analysis	23
Description	25
Gui Design Discussion	26
Main menu- building	26
Building a Building - initial state of the basic flow of events	27
Choosing a building (step 6)	28
Construction in progress (step 11)	29
After the construction (final step)	30
Main menu for training	32

Train and chill units - initial state of the basic flow of events	33
Train Units training time in the queue (step 3)	34
After the training - while units are moved to army camp (final stage)	35
Initiate an Attack and Apply Battle Result - initial state of the basic flow of events	37
Choosing the type of attack (step 2)	38
Choosing the random village from the same League for attack (step 4)	38
Battle time (step 7)	39
Victory and defeat page (step 13)	40
After the attack - single attack (final stage)	41
Choosing the village from the opponent clan (step 2a3)	42
Join clan with ban- initial state of the basic flow of events	42
Invitations from clans	44
Successfully joined player to clan (step 5)	45
After the positive admission to the clan(final stage)	46
Unsuccessfully joined to clan (Step 3b1)	47
Unsuccessfully joined to clan (Step 3a)	47
Search Clans	48
History of the memberships in the clans	49

User Requirements

The Clash of Clans system is designed to support both players and clan leaders in managing their villages, armies, clans, and battles. The system allows players to build and upgrade their villages, train troops, join clans, and participate in wars. Clan leaders and co-leaders are given special permissions to manage the clan, including recruitment, war strategies.

Each Player has a profile with basic information such as username, level, trophies, and league. Players can build and upgrade Buildings in their Village, train different types of Troops, create different spells, and engage in Battles against opponents. A player can belong to only one Clan at a time, while each clan can consist of multiple players. We want to store the

information of the player's membership and past memberships to make sure that the clan will not admit the banned user.

There can be two types of Villages (regular, dark) consists of multiple buildings, which are divided into subcategories: Resource Buildings (Gold Mine, Elixir Collector, Dark Elixir Collector), that have a production rate, type and maximum storage capacity. Defensive Buildings (Cannon, Archer Tower) have damage points per second, type, range and target (air, ground). Army Buildings (Barracks, Army Camp) have their own troops capacity and type. Buildings can be upgraded to higher levels, with increasing costs and construction times. Each building has hit points, build time, and resource cost. Buildings cannot exist independently of a village.

All units are trained in the Barracks and rest in army camp. Each unit troop has certain attack style and by default not available once per player. Also, there can be only certain number of units in village and while they are in village have current hit points, current level, time till construction and location. Units are divided into categories: Elixir (barbarians, Archers, Giants, Goblins, Dragons) and Dark Elixir (Hog, Minion, Witch, Valkyrie, Golem). All units must belong to at least one type in each aspect. Also, units can be either air or ground attacker

Heroes (such as Barbarian King, Archer Queen, and Grand Warden) are special units that have unique abilities, regeneration times, upgrade systems, but unlike regular troops (which are also another type of unit), they can only exist once per player.

Spells represent another category of assets that can be created by Player. Each spell has effects (healing, rage, freeze), resource costs, and durations. Spells are optional assets for a player, but not all players have to use it.

Clans are central to the system. A clan has a name, badge, description, and a set of members. Each clan may participate in Clan Wars that have duration, reward and result (win or lose), which involve coordinated battles against another clan. When player joins to the clan, he receives a role (Leader, Co-Leader, Elder, Member) and join date.

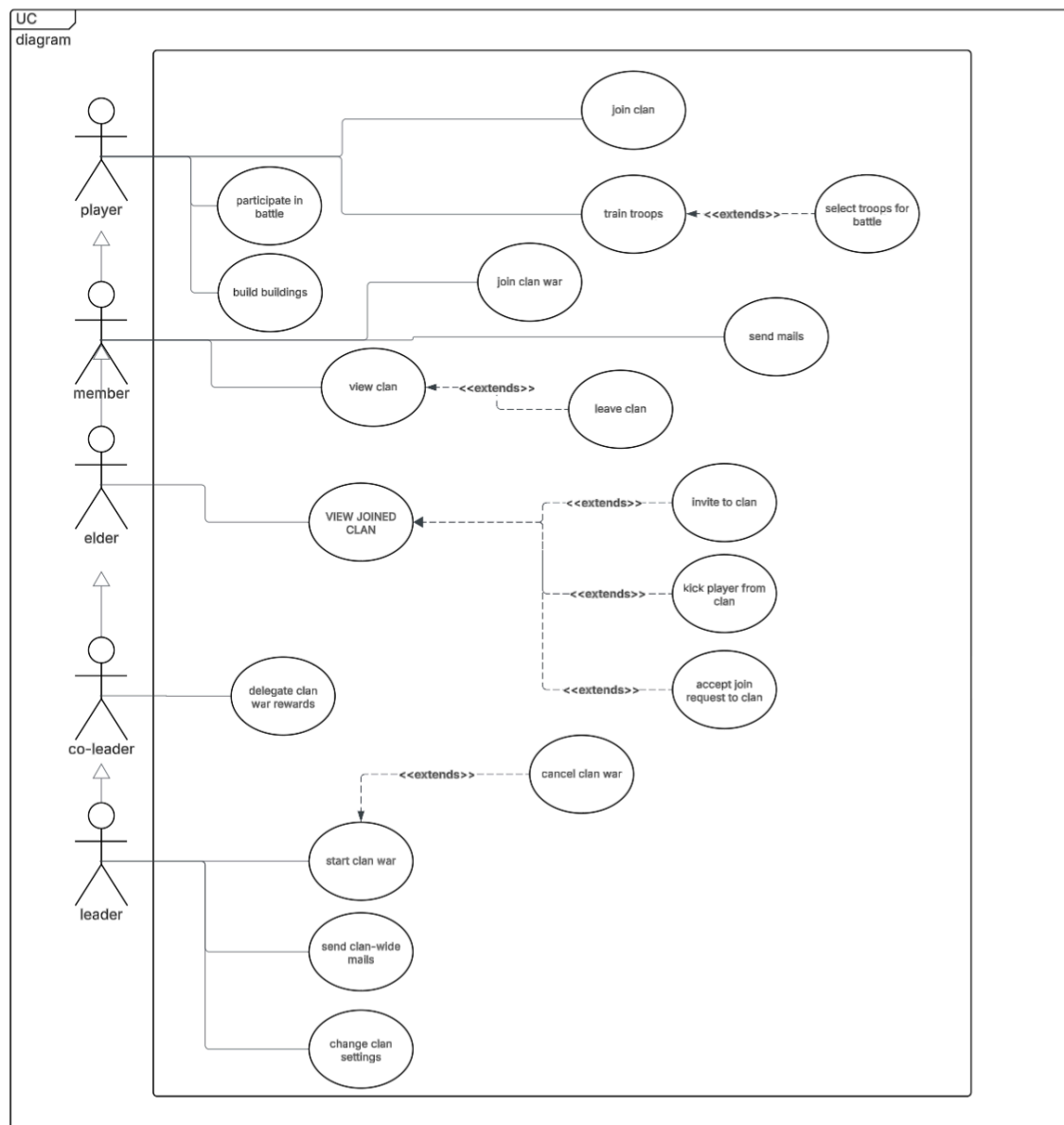
The system tracks Battles between players, where a battle has attributes such as time, stars earned and loot gained. A player may participate in multiple battles, and each battle links exactly two villages/players that one is attacking and the second is being attacked. Battles are ordered so that there is a sequence of attacks during a war, differentiated by timestamp. Battles also has two types single and clan type (clan war - each player has 2 attempts per war). You can earn trophies during a battle that will be calculated based on level of the opponent and stars gained on battle

3-CLASH OF CLANS

Achievements are awarded to players when they reach specific milestones . Each achievement has a name, type, description and reward.

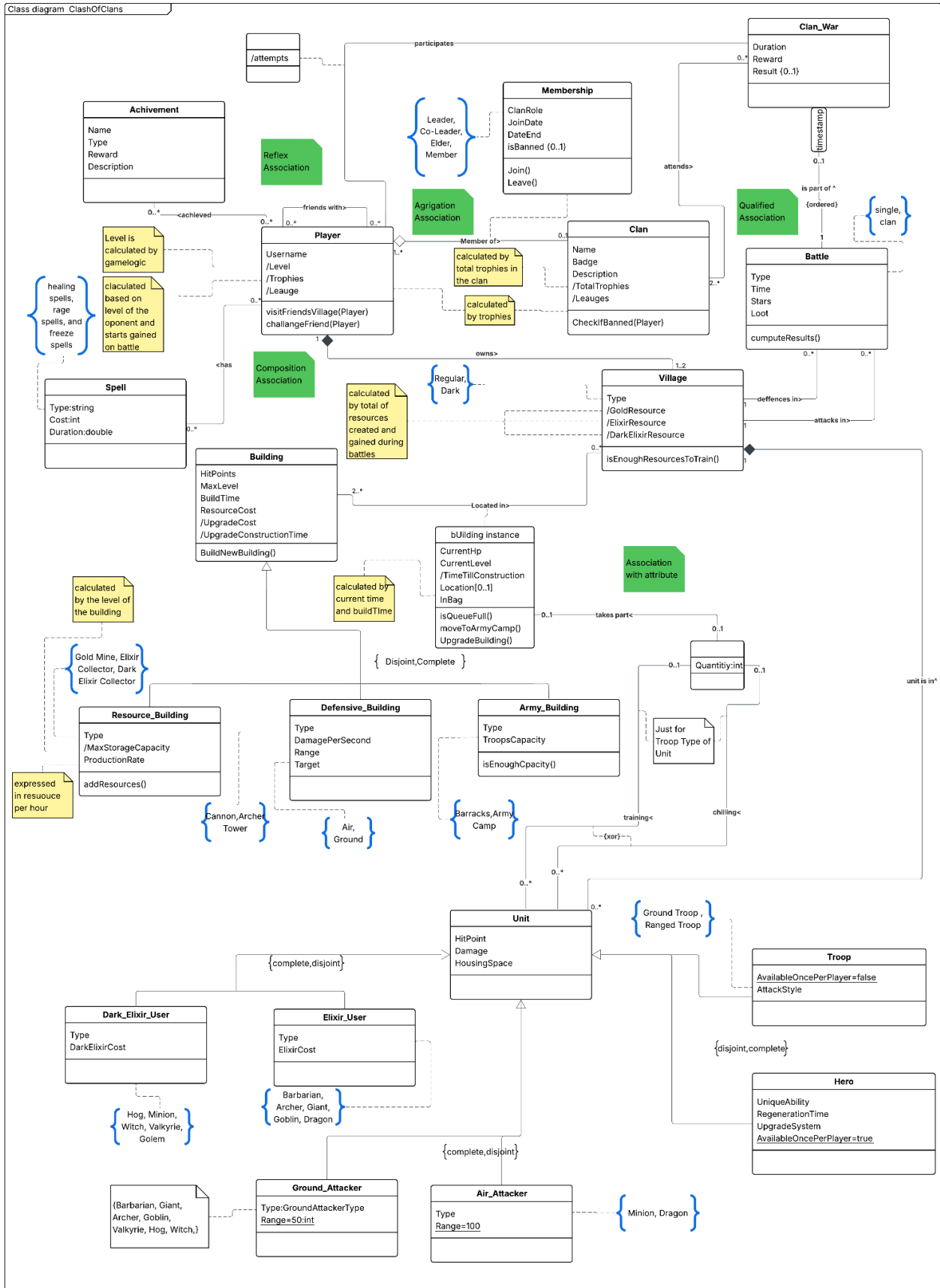
The system includes Friendship between players. A player can add another player as a friend, enabling friendly challenges and visits to their villages.

Use-Case Diagram



Class diagram - Analytical

Class diagram ClashOfClans



Description

The main entity is the Player, who owns a Village/Villages (max 2) made up of different Buildings, including resource buildings (like gold mines), defensive buildings (such as cannons or archer towers), and army buildings (like barracks and army camps). Players can train Units, which include Troops and Heroes, each with attributes such as hit points, damage, and housing space. Troops may be ground or air attackers and use different resources like elixir or dark elixir. It's worth showcasing that the Troop can either train or rest in the building. Players can join Clans, forming social groups through the Membership relationship, which defines roles, levels, and join dates. We need to store information of the player's membership to not add the player that is banned in a given clan. Clans compete in Clan Wars, earning rewards and trophies based on their performance. Players can also engage in Solo Battles with other villages to gain loot, trophies, and experience. The diagram includes Spells that influence battles and Achievements that track player progress and offer rewards.

Class diagram - Design



Design Decision

[Design Decision HERE]

The project is developed using Java language with an object-oriented programming approach that organizes the game world into clearly defined classes representing buildings, soldiers, resources, and game control mechanisms. The design diagram presents the structure of the system as it is defined for implementation in Java. Each class is shown with its attributes, including their specific Java data types and any enumeration values used. It outlines elements such as players, clans, buildings, resources, soldiers, and units and how their information is represented at the code level. Combat and movement behavior is defined through structured logic that determines targeting, damage application, and positional updates. The algorithms controlling these actions ensure predictable and balanced results based on defined troop and building parameters. Object creation for troops and buildings is managed through organized constructors or factory-style methods, supporting scalability and adding new game elements with minimal structural changes.

Java types such as String will be implemented for text for numerical values int or double. Boolean for the nullable or false true values, and also we use LocalDateTime while we speak about dates. Maps, Arrays, Lists and enums are also used .

The method of saving data is implemented for now in one of the classes (as methods save and load to binary file .dat) and Java serializable, we have implemented methods like save and load also delete in static class to implement it in each necessary for that reason class. file format - .dat can be also saved in .ser, .txt which gives flexibility. Everything is achieved with class extent.

Attribute Validation

The system achieves attribute validation primarily through procedural checks implemented within the domain object's constructors and setter methods, rather than relying solely on external declarative attributes. Additionally, framework-specific tools like the @DateTimeFormat annotation are used for data binding and formatting of time-based attributes.

Optional Attributes

The system uses optional attributes to accurately model object states where data is not always available or needed .Examples include the Results attribute in ClanWar, which is null until the war concludes, and the Location attribute on a Building instance, which is null until the player places the building on the map.

Multi-value attributes

The system utilizes various Java collections (such as ArrayList , HashSet , Array, Map) to implement multi-value attributes. For instance, a HashSet is employed to ensure that a collection of objects contains only unique values, providing automatic data integrity and preventing duplicates. A Map is used for attributes that require a key-value storage mechanism. ArrayList attributes are utilized when the requirement is to store an ordered or unordered sequence of all records for a specific class or attribute.

Derived Attributes

The system implements Derived Attributes – values that are calculated dynamically based on the state of other attributes .For example in our system we implemented method called calculateMaxStorage In resource building class that calculates maximum storage based on level of building instance, in village class resource calculated by total of resource created and gained during battles , clan and player class league calculated by trophies , in player class level calculated by game logic.

General association implementation

In our system association was implemented using reverse connections to ensure correct logic by requiring that when an object is added or removed to a collection on one side of a relationship, the corresponding reference or collection on the linked object is simultaneously updated. If we want to add a new object to our system first we check for exceptions (like if the added object is null , has some incorrect attributes or has already been added to the system to get rid of recursion).Also if we want to store some object in constructor of some class to maintain reverse connection we implemented checks for it inside the constructor (check for null value , or some logical exceptions such banned player). Same checks were implemented for remove methods.

Composition

In our system, the Composition association was implemented between the Player and Village, where the existence of one object is entirely dependent on the other. This is enforced because every Village object is created and initialized directly within the Player constructor, meaning a village cannot exist without an owner. When a PLayer object is deleted, it automatically calls the delete() method on all its associated Village objects, ensuring that the parts cannot persist if the owning object is removed. Also Composition implemented between the Village and the Unit to represent mandatory ownership of troops. This relationship is enforced structurally because the Unit constructor strictly requires a Village reference, immediately throwing an exception if ownership is not provided.

Qualified associations

In our system, the Qualified Association was implemented in the ClanWar class through the use of the battlesInClanWar attribute. This attribute is defined as a Map<LocalDateTime, Battle> ,

where the LocalDateTime of the battle acts as the qualifier. This structure allows a specific Battle object to be quickly retrieved by using its unique time attribute as the key.

Association class

The Membership class acts as an Association class to manage the many-to-many relationship between Player and Clan . Membership accepts one object of Player and Clan (also check if these values are null) within its constructor to ensure that a valid association is immediately formed.

Reflex Association

The Player class demonstrates a Reflexive Association through its management of the friends list. When one Player(Player A) wants to add another(Player B) to the friend list the same method calls in Player B to add a Player A. This establishes the link in both directions. Also the system checks for duplicates to prevent self-friending.

Association validation

To validate associations in our system we check if the object (that we want to add or remove) is null , if the adding or removing method meets some logical exception (like banned players) , also check if there is a duplicate in the system to get rid of recursion.

Inheritance Implementation Strategy (Units)

The system features Multi-Aspect Inheritance involving three aspects: Role (Troop/Hero), Attack Domain (Ground/Air), and Resource Type (Elixir/Dark Elixir). Since Java does not support multiple inheritance, we implemented this using the Flattening strategy for the Attack and Resource aspects, while maintaining standard inheritance for the Role aspect.

Flattening of Attack and Resource Aspects: Instead of creating separate classes for GroundAttacker, AirAttacker, ElixirUser, and DarkElixirUser, these distinctions were flattened into the abstract Unit class as attributes (AttackDomain and ResourceKind enums). Logic specific to these types (like Range) is handled via delegation to the Enums within the base class.

Full Flattening of Cost Structure: The abstract Unit class flattens the cost structure for all entities. Instead of creating distinct classes for cost types or isolating costs in subclasses, the Unit class contains both elixirCost and darkElixirCost fields. The constructor ensures that only the cost corresponding to the unit's ResourceKind is set, while the other remains null. This ensures that both Troops and Heroes have a consistent data structure regarding their resource consumption.

Safeguards: To maintain data integrity in this flattened structure, the Unit constructor includes a validateResourceCompatibility method. This safeguards against invalid combinations (e.g., creating a Ground unit that is logically supposed to be Flying) as recommended in the implementation notes.

Inheritance Strategy (Buildings)

The Building class and its subclasses use standard inheritance with a disjoint and complete structure.

Disjoint: Subclasses (ArmyBuilding, DefensiveBuilding, ResourceBuilding) have distinct, non-overlapping roles and attributes (e.g., Damage for defense, ProductionRate for resources). Java's single inheritance naturally keeps them separate.

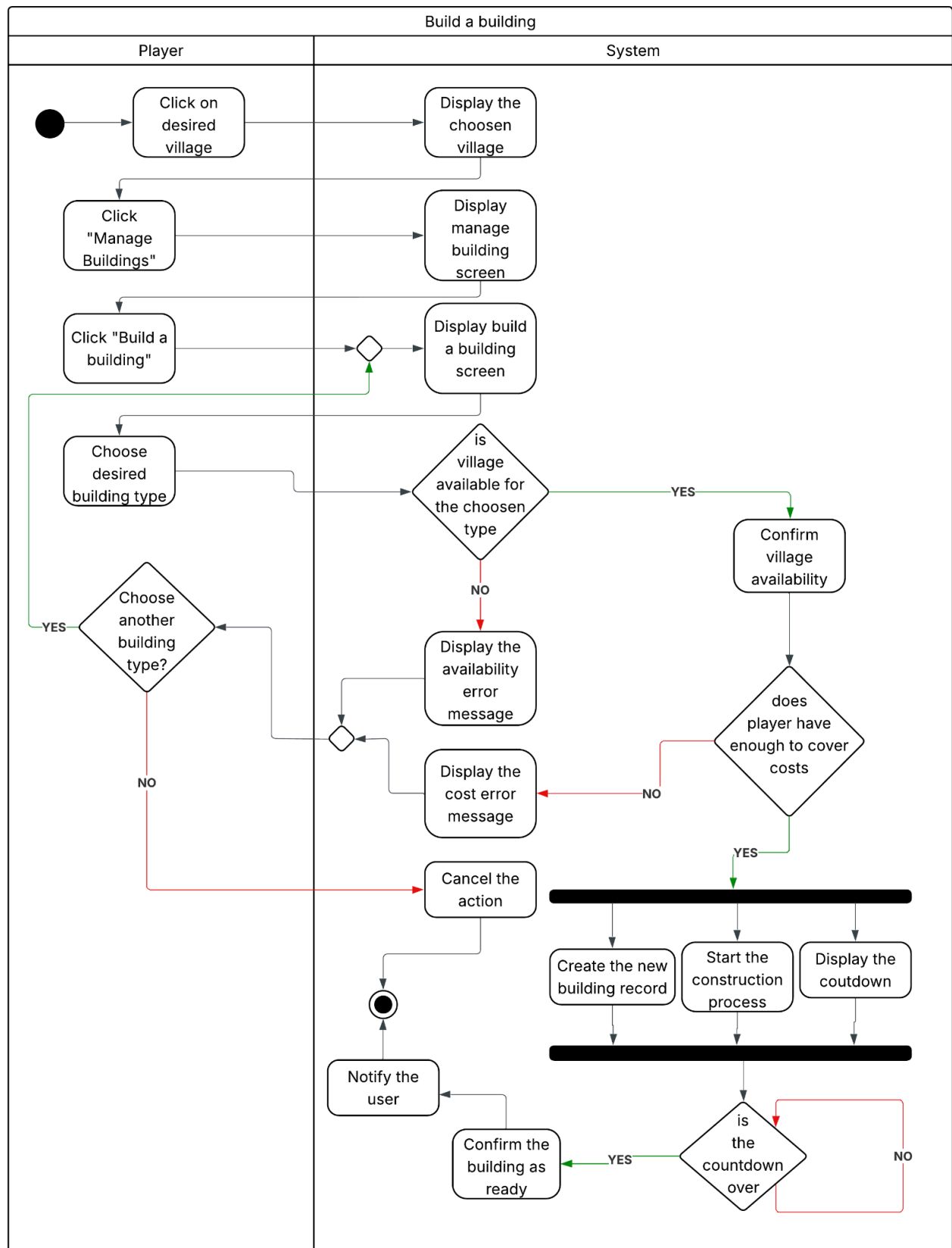
Complete: The Building class is abstract, so you cannot create a generic building. Every building must be a specific subclass.

Why not flatten?

Combining all fields (like storage and damage) into one class would break the Single Responsibility Principle and leave many unused/null fields, making the class messy.

Use Case and Activity Diagram

1:Building a Building



Building a Building

Actor : Player

Purpose and context: A player wants to create a new building into their village

Assumption: 1- Player owns the village where they want to add building

Basic flow of events:

- 1- The player clicks on the village where they want to add a building
- 2- System displays the choosen village
- 3- The player clicks on the "mannage buildings" button
- 4- System displays manage buildings screen
- 5- The player chooses "build a building"
- 6- System displays build a building page
- 7- Player chooses the type of the building they desire
- 8- System confirms the village availablity
- 9- System deducts building costs from the player and adjusts village availabilities
- 10- System creates the new building record , starts the construction process and displaying the coutdown to when the building is ready to use
- 11- System comfirms that building is ready to use and notifyies the user when the countdown is over

Alternative flow of events:

Village is not available

- 8a1- System displays "Village is not available for this building type!! Try another building type or cancel the action."
- 8a2- Player chooses to build another type of building
- 8a3- Return to point 6

Player decides to cancel due to availability

- 8a2a1- Player chooses to cancel the action
- 8a2a2- System cancels the action

Not enough resource to cover the costs

- 9a1- System displays "You do not have enough resources to cover the costs!! Try another building or cancel the action."
- 9a2- Player chooses to build another type of building
- 9a3- Return to point 6

Player decides to cancel due to costs

- 9a2a1- Player chooses to cancel the action
- 9a2a2- System cancels the action

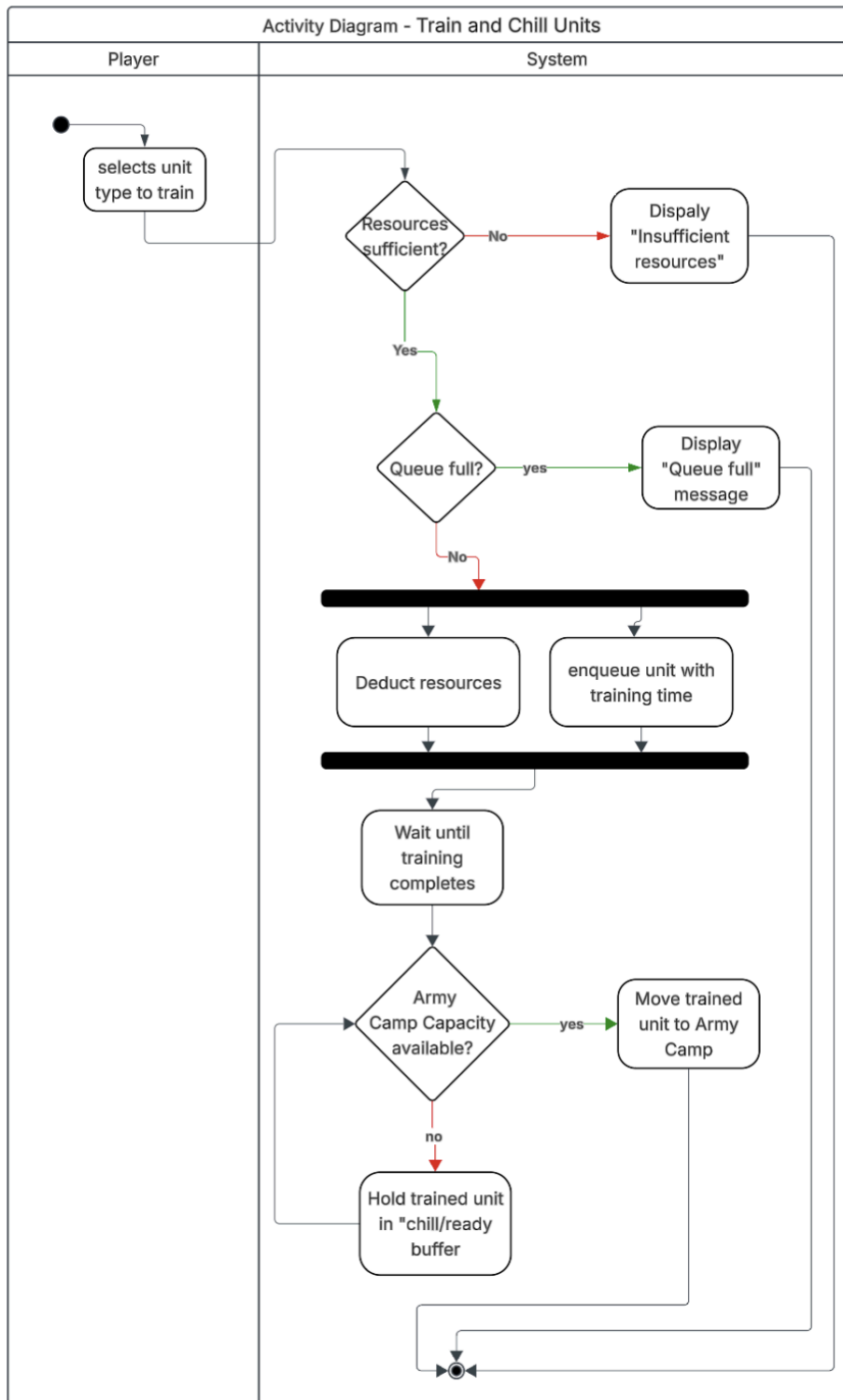
Post condition:

Basic: New building created and ready

Player decides to cancel due to availability: Building a building request gets canceled

Player decides to cancel due to costs: Building a building request gets canceled

2: Train and Chill Units



Use Case: Train and Chill Units

Actor: Player

Purpose and Context:

Allow the player to train units (troops or heroes), consume resources, manage training queues

Assumptions:

1. The player has enough resources to train at least one unit.

Preconditions:

1. Army Camp capacity is defined.

Basic Flow of Events:

1. The player selects a unit type to train (e.g., ground or air troop, elixir or dark elixir user).
2. The system checks available resources and current queue size.
3. If valid, the system deducts the required resources and enqueues the unit with its training time.
4. When training completes, the system attempts to place the unit in the **Army Camp**.
5. If the camp is full, the trained unit is held in a **"chill" or ready buffer** until space becomes available.
6. When capacity frees up, the system automatically moves chilled units from the buffer into the Army Camp.

Alternative Flow of Events:

- **Insufficient Resources:**

- 2a. The system rejects the request and notifies the player of the missing resources.

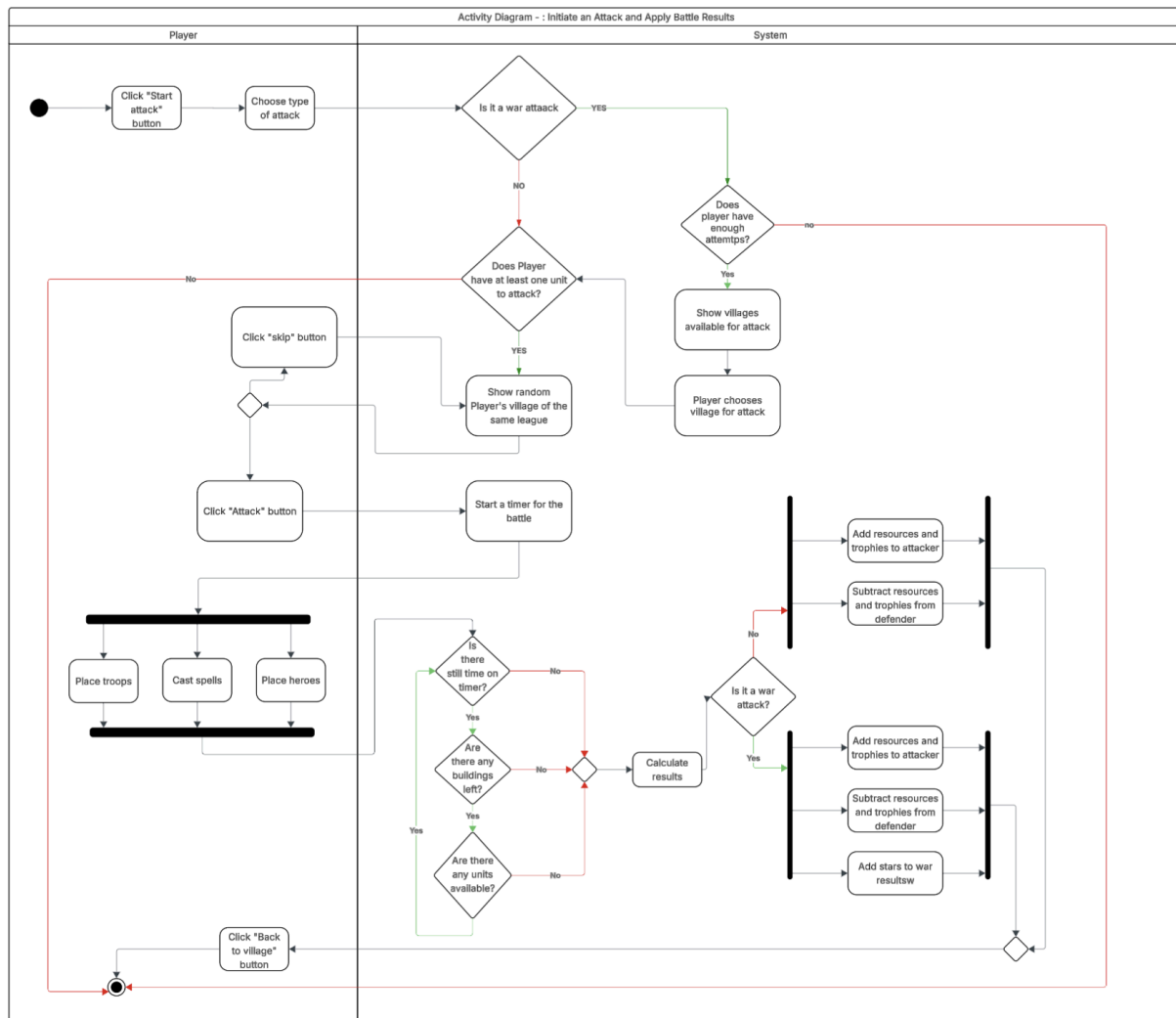
- **Queue Full:**

- 2b. The system rejects the enqueue request and suggests clearing or expanding the queue.

Postconditions:

- **Successful training:** Queue updated, resources consumed, trained units stored or held in the chill buffer.
- **Rejected training:** No resources deducted, no queue or buffer changes.

3:Initiate an Attack and Apply Battle Result



Use Case: Initiate an Attack and Apply Battle Results

Actor : Player

Purpose and context: Let a player start an attack (single or clan war). After the attack ends, ingest the combat log and apply results (stars, loot, trophies/war score).

Assumptions: empty

Precondition:

1. If war attack: war is active, attacks remaining > 0.

Basic flow of events:

1. Player clicks "Start an attack" button.
2. Player chooses "single" of attack.
3. System confirms that Player has at least one troop/spell/hero ready for attack.
4. System shows random village of Player with the same League as attacker.
5. Player confirms attack.
6. System starts an attack for Player and sets a timer.
7. Player places troops/spells/heroes.
8. System checks if the time has gone.
9. System checks if there are still buildings in the defender village.
10. System check if there are still troops/spells/heroes available for the attacker.
11. System ends the battle and calculates the result.
12. System applies data: attacker resources and trophies gained, defender resources and trophies lost.
13. System shows the result screen to the Player.
14. Player clicks button "Back to village".

Alternative flow of events:

Player chooses "war" attack.

- 2a1. Player chooses war attack.
- 2a2. System check if Player has enough attempts to attack.
- 2a3. System shows villages available for attack.
- 2a4. Player chooses village to attack.
- 2a5. Continue step 3

Player skips showed village in single attack.

- 5a1. Player click "skip" button.
- 5a2. Back to 4 step.

Player doesn't have enough attempts.

- 2a2a1. Get to the end.

Applying results if a war attack.

- 12a1. System applies data: attacker resources and gained, defender resources and trophies lost, adds amount of gained stars to war results.
- 12a2. Continue step 13.

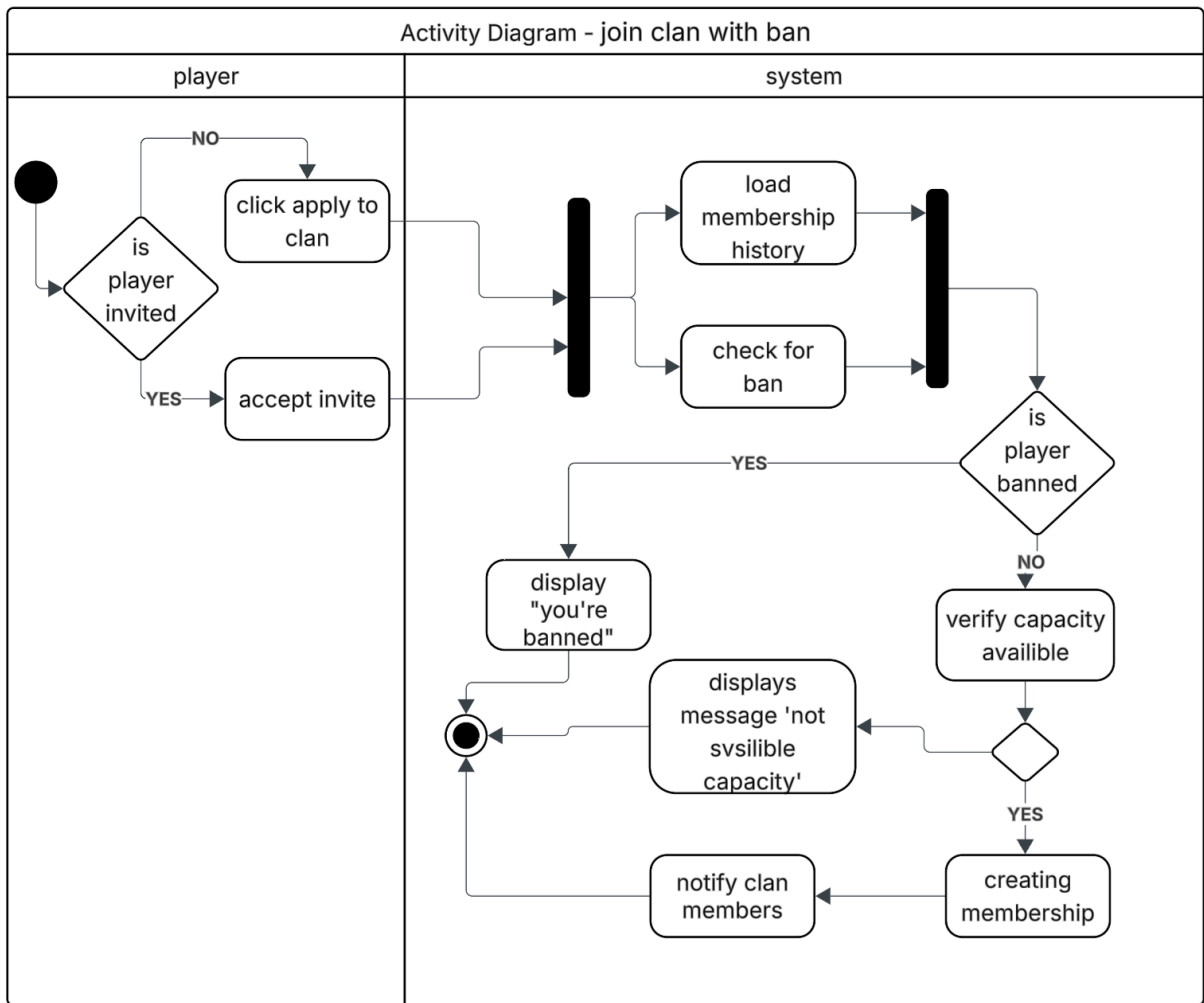
Post condition:

Basic: **Battle** ended. Player resources/trophies/achievements updated; war timeline updated.

Player doesn't have enough attempts: no change in resources.

Player doesn't have enough units: no change in resources.

4:Join Clan With Ban



Use Case: join clan with ban

Actor : Member / co-leader

Purpose and context : A player a clan while the system enforces ban and rejoin cooldown .

Assumption:

1. The player is no currently in any clan .
2. Clan recruitment is open or an invitation is issued .

Precondition: **None**

Basic flow of event :

1. The player applies to the clan or accept the invite .
2. The system loads MembershipHistory and clan ban rules.
3. The system verifies: not banned, capacity available.
4. The system creates Membership.
5. The system notifies clan members and updates clan member count.

Alternative flow of events:

Player is banned

3a. The system rejects the request with status "BANNED"; no changes occur.

Capacity not available

3b1 The system rejects the request with status "not available capacity"

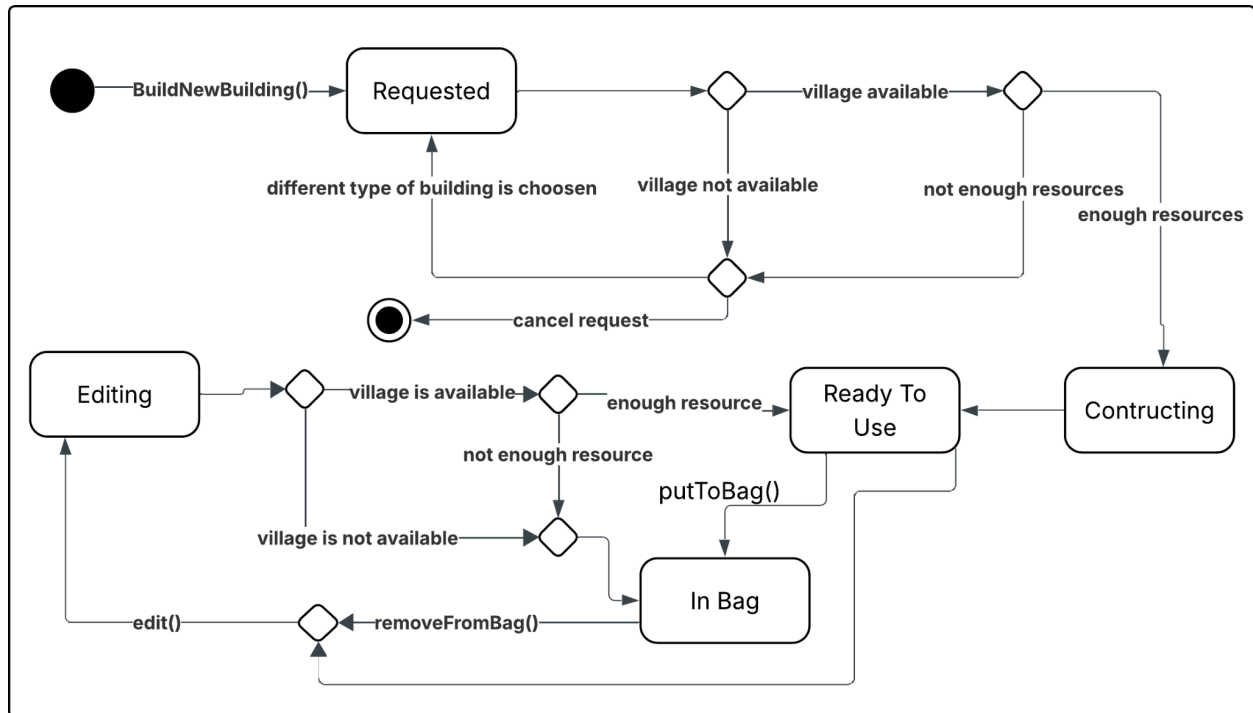
Post condition:

Basic: Player is a clan member .

Rejected: No changes to memberships.

State Diagram

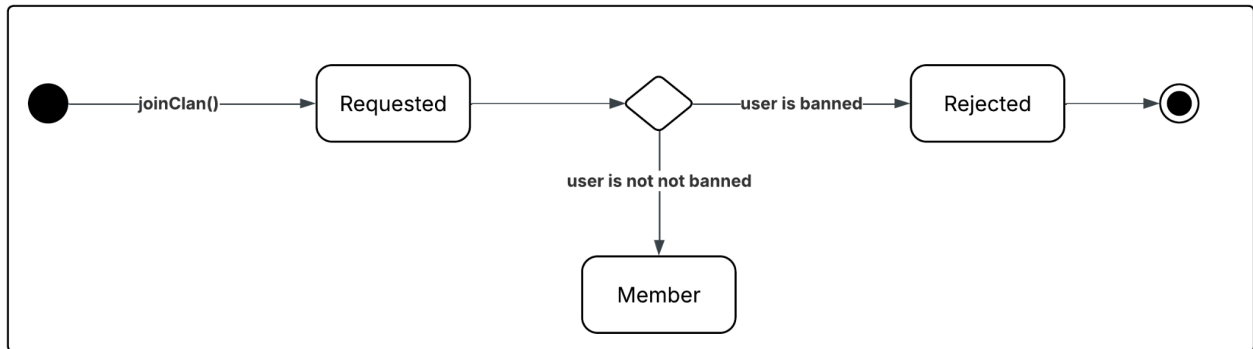
Building State Diagram



Description

Creating a new building process starts with BuildNewBuilding(), with that our building is in the state Requested. Buildings stay in that state until village availability and the fact that user has enough resources is confirmed, if the confirmation has errors then the request gets canceled. After the confirmation the building enters Constructing stage and stays till it is ready to use. A building stage Ready To Use can be put into a bag or can be edited by its owner user. A building in bag needs to get into the Edit stage in order to be back in the map and be Ready To Use.

Join Clan Player



Description

When a player tries to join a clan, system first checks if the user has been banned before in that clan, if the user hasn't been banned then user becomes a member. If user has been rejected , their request record gets deleted.

Changes to the Class Diagram After Dynamic Analysis



Description

The Class Diagram after dynamic analysis introduces a new way of implementing inheritance .

Now Unit class is abstract and uses a flattening approach to handle a multi-aspect inheritance , 4 new sets of enums(DarkElixirUserTypes, ElixirUserTypes, GroudTypes, AirTypes) were added to Unit class , Type became UnitType instead of string , added domain attribute to store values of AttackDomain enum (that stores static values such as ground and air) , resourceKind that stores value of ResourceKind enum , ElixirCost which is expressed as final value in enum AttackDomain attribute of an integer that represents the cost of a troop and also DarkElixirCost that represents the cost of a troop in dark elixir.

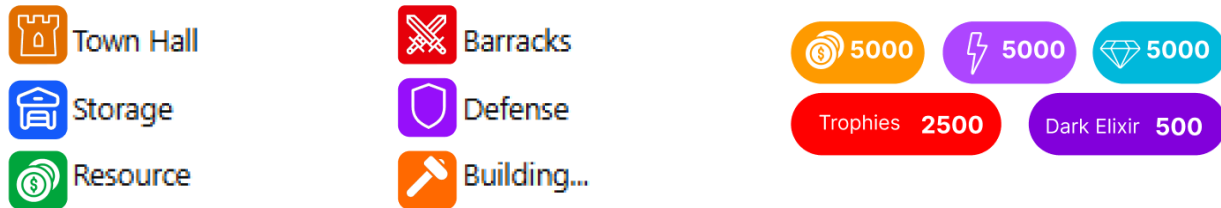
With use of composition we implemented Troop and Hero classes that became a subclasses of Unit class, Unit can either Troop or a Hero at one time. They still are disjoint and complete (usage of simple java extends)

Also we added new association design between unit(trop) and the building instance -> now the quantity (Barack and armycamp queue logic is filled according to the well structured and simpler model)

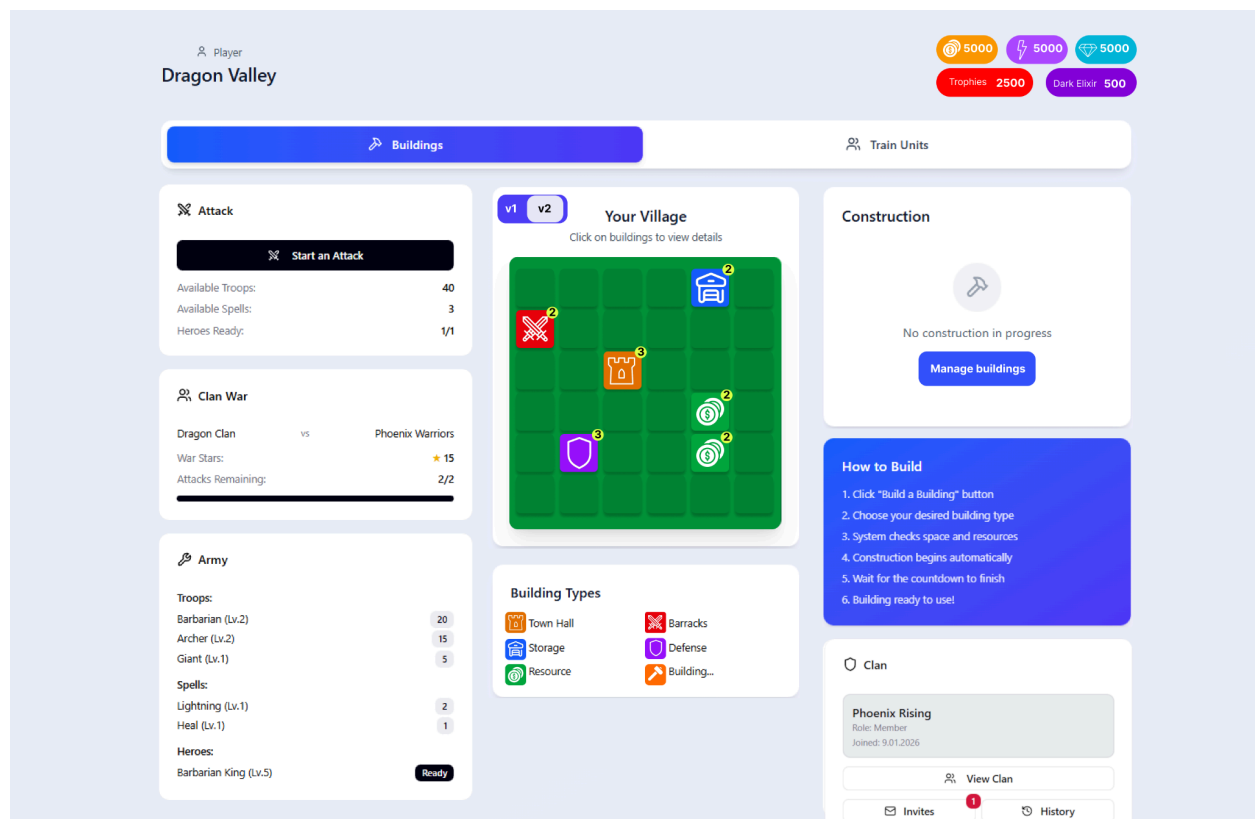
For inheritance building and its child classes we use simple approach as extends- Java with building classes now being abstract and all its subclasses use method super in the contractor to call building class constructor.

Gui Design Discussion

Gui design is done with the simple and modern approach- app has simple shapes - no many texters to avoid overcomplication. Each functionality and attribute on every single page is designed to have either different colors or icons illustrating its purpose. For example building types and the resources of the player:



Main menu- building



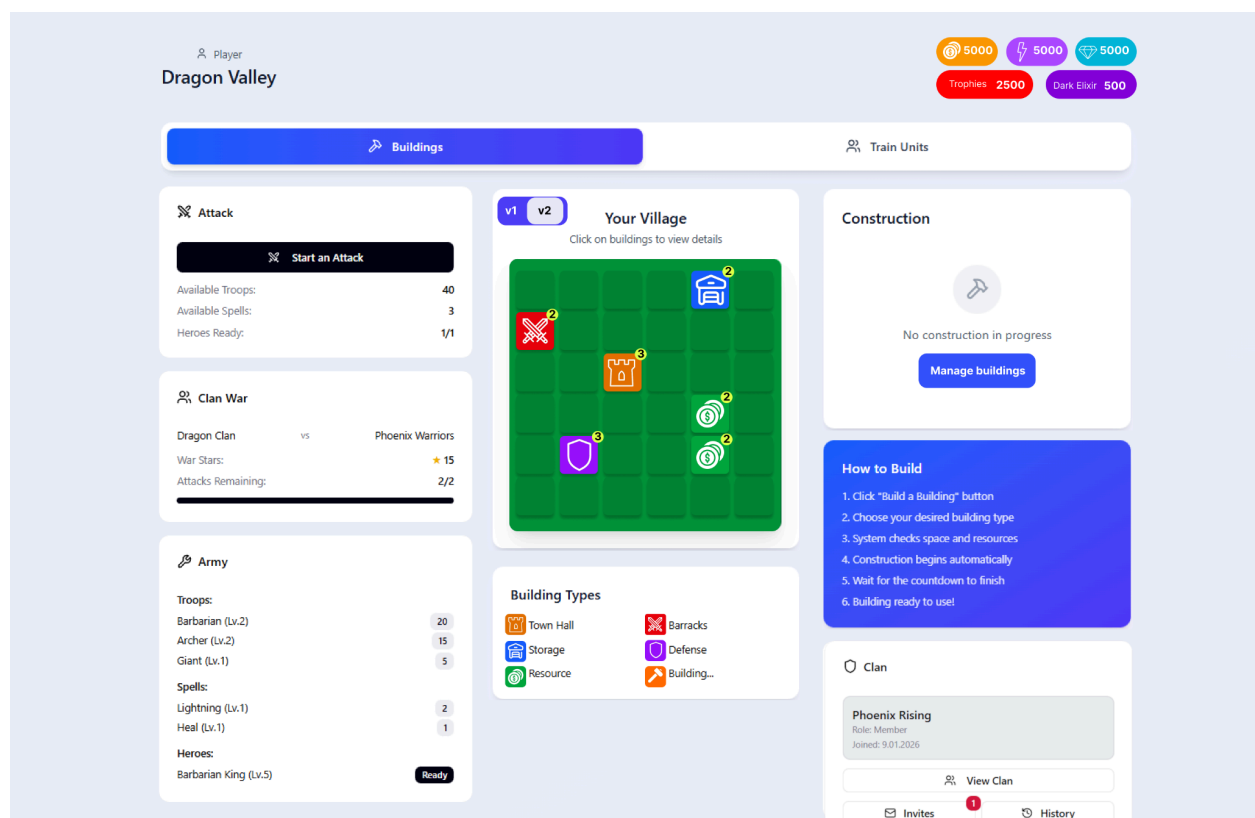
The screen above is the main menu of the building, we can find the informations like:

- the clan (if we are in one or find new one)
- Instruction how to build a building
- Army that the player has
- Ongoing clan war
- Current resources of the player
- Ongoing constructions
- And the village with its buildings of different types and levels

Also we can find functionalities like:

- Changing the menus
- Starting an attack
- Manage buildings
- Join clans or see invitations with the history

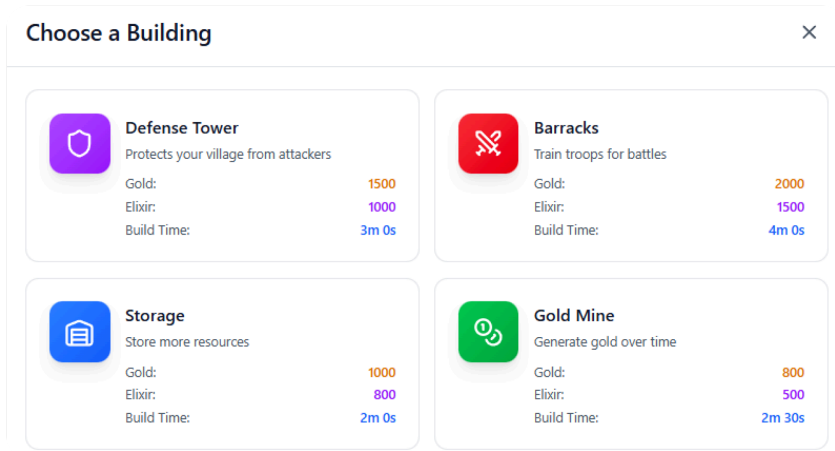
Building a Building - initial state of the basic flow of events



Before the adding the building/ building it there are no constructions in progress. This part remains empty till the point when the player decides to build it. (Step->1) needs to click whether wants to do it on the village v2 or v1 (this position is adaptable only if the player has two villages) then while clicking on "Manage buildings" will be navigated to the desired page. They can choose to build a building and choose the building they desire. System confirms the

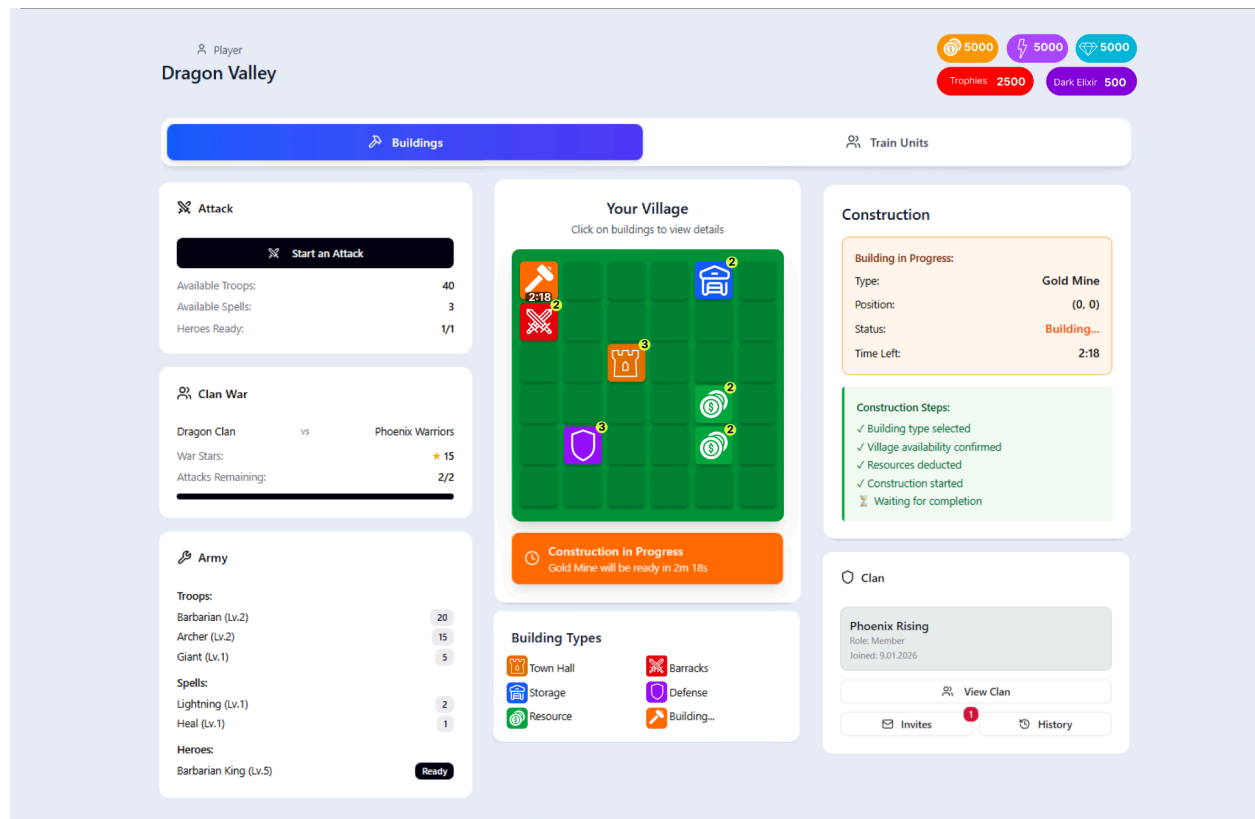
availability and proceed with the deducting the costs and creating new record (with the countdown)

Choosing a building (step 6)



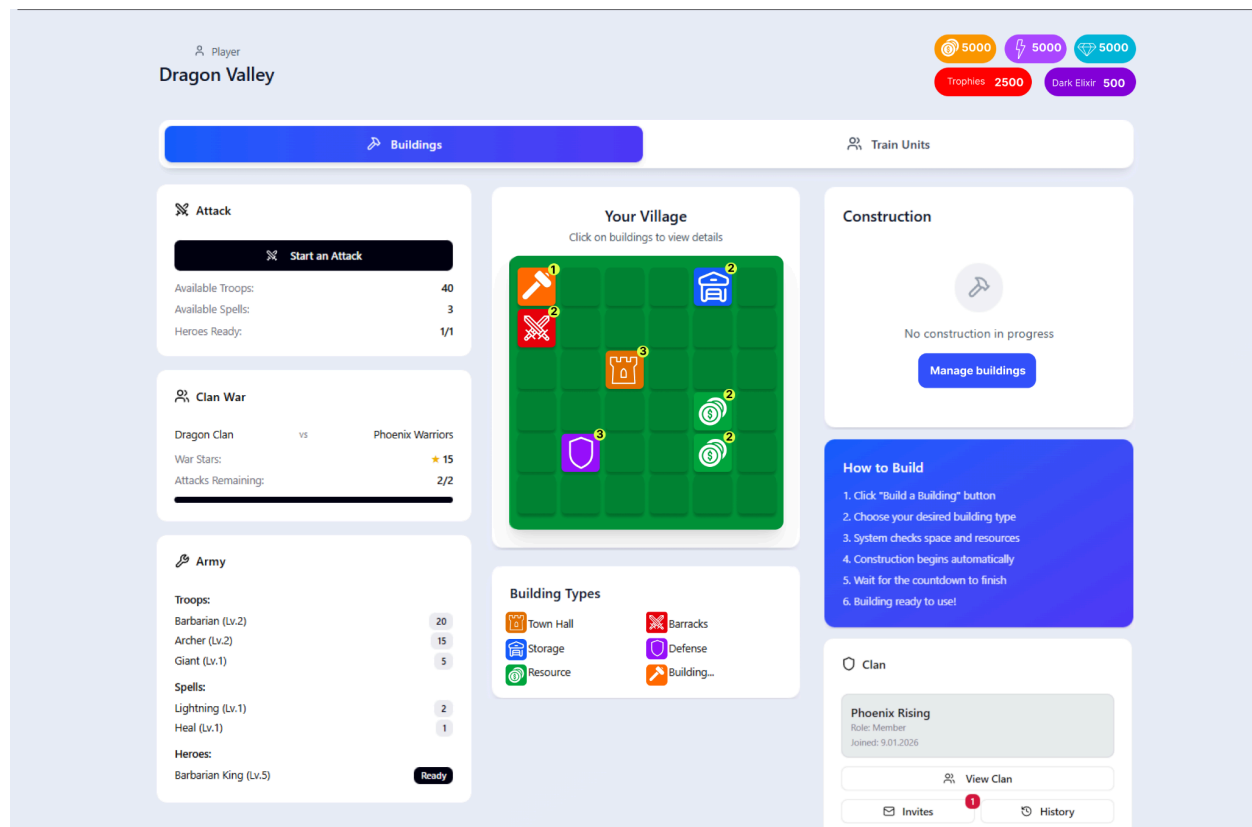
The screen illustrated above shows us the page where player can decide which type of building wants to build. (It is written there besides the type of building resources cost and build time). The player can always choose to cancel the operation by clicking the "X" button. No change would appear in the system at this time.

Construction in progress (step 11)



The above screen illustrates the main menu of the building - player can see that the construction is in the progress (right window), also there is information of the position. The village that is in the middle has the new component with the icon of the building (of the concrete type) with the timer till the end of the building process. The resources were deducted from the player's account.

After the construction (final step)



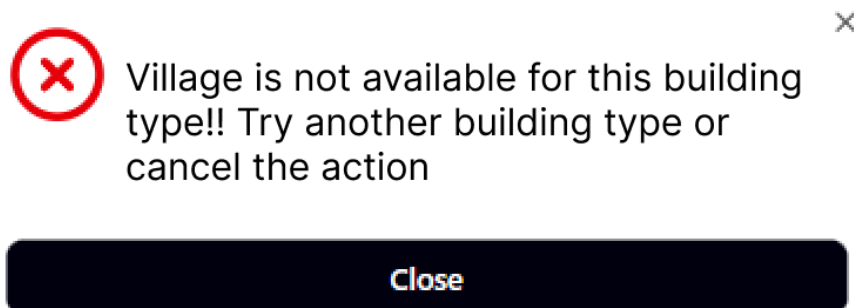
After the construction process the building appears in the village with the level 1 and the constructions in progress again are empty.

As the alternative flow:

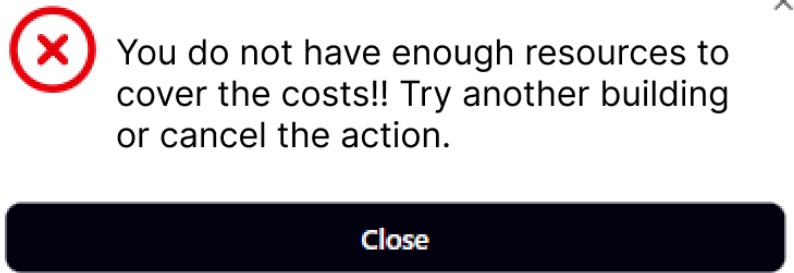
(step 8a1) the building type that the player wants to build is not correct, then the player needs to decide of the new type of building (**redirection to step 6**)

otherwise:

(step 8a2a1) player may decide to cancel due to availability. No changes would appear in the village then. **Pop Up message below - and redirection to main menu building**

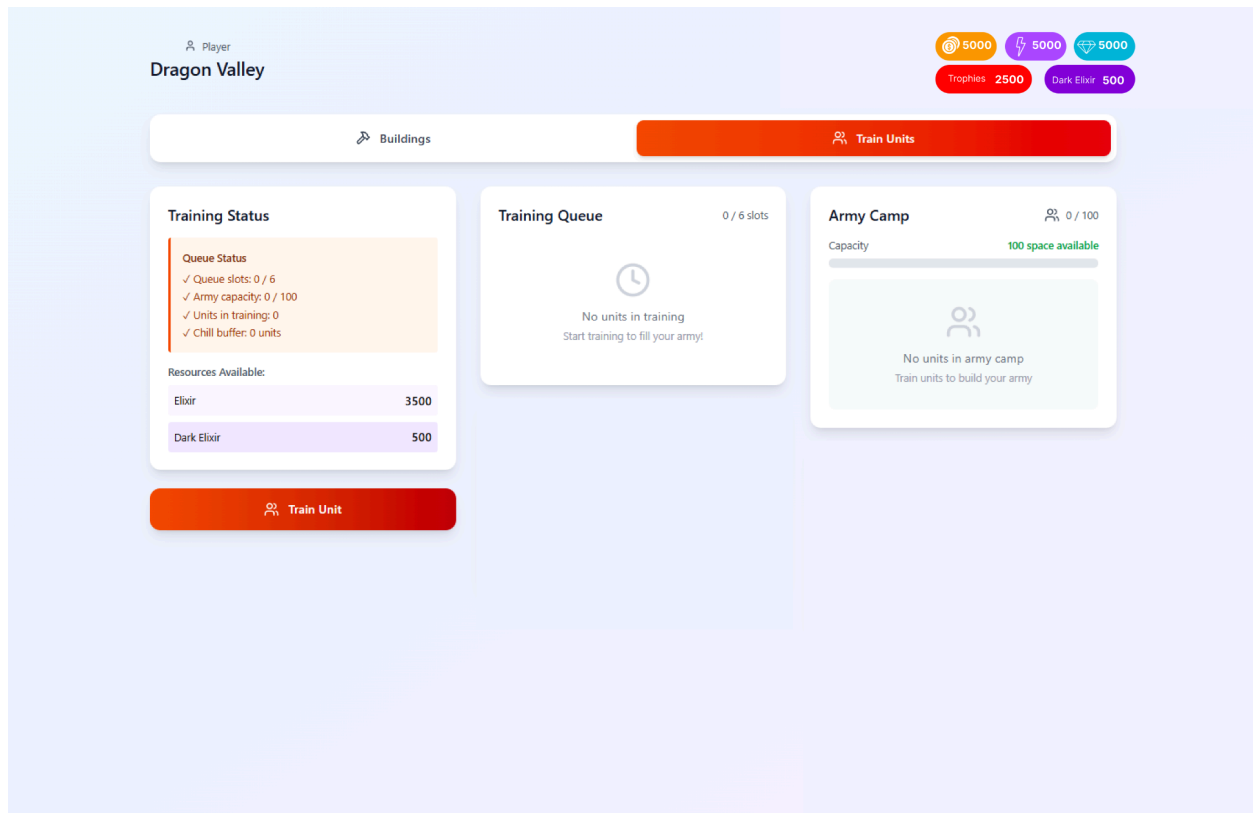


(step 9a1) player does not have enough resources to cover the costs (bellow pop up window would appear), then player can choose another type of building (**redirection to step 6**)



(step 9a2a1) player cancel due to costs. No changes would appear in the village then.
(redirection to the main menu -building)

Main menu for training



The screen above illustrates the menu where player can see all events about Training the Units. (To change the menu from Building to training is needed just to click on “Training Units” button) Before the training of the unit (troop) we have an empty list with training queue and army camp doesn't have anybody there (initial state - we can have more in army camp, but we must have a limit of units established). After clicking the “Train Unit” Button the player is navigated to the selection page where the basic flow of events starts for training and chill units.

Train and chill units - initial state of the basic flow of events

Train Units

Select a unit type to begin training



Barbarian
Ground • Elixir
A fearsome warrior with a mohawk

Cost:	80
Time:	20s
Space:	1



Archer
Ground • Elixir
Ranged attacker with great aim

Cost:	100
Time:	25s
Space:	1



Giant
Ground • Elixir
Big and tough, ideal for soaking damage

Cost:	500
Time:	45s
Space:	5



Wizard
Ground • Elixir
Devastating area damage dealer

Cost:	400
Time:	60s
Space:	4



Dragon
Air • Elixir
Flying beast with powerful fire attacks

Cost:	1200
Time:	120s
Space:	20



Minion
Air • Dark Elixir
Fast flying unit

Cost:	40
Time:	30s
Space:	2



Hog Rider
Ground • Dark Elixir
Fast moving troop that targets defenses

Cost:	100
Time:	60s
Space:	5

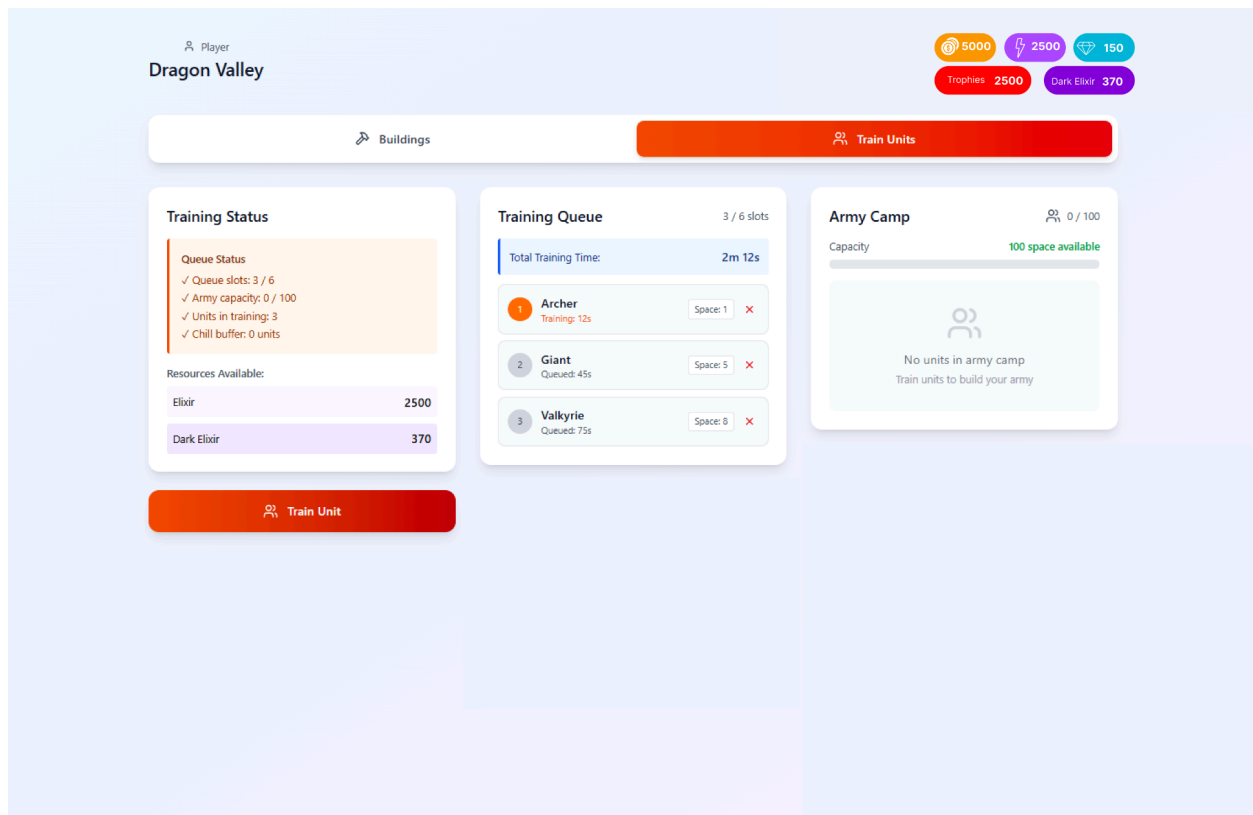


Valkyrie
Ground • Dark Elixir
Spins to attack multiple enemies

Cost:	130
Time:	75s
Space:	8

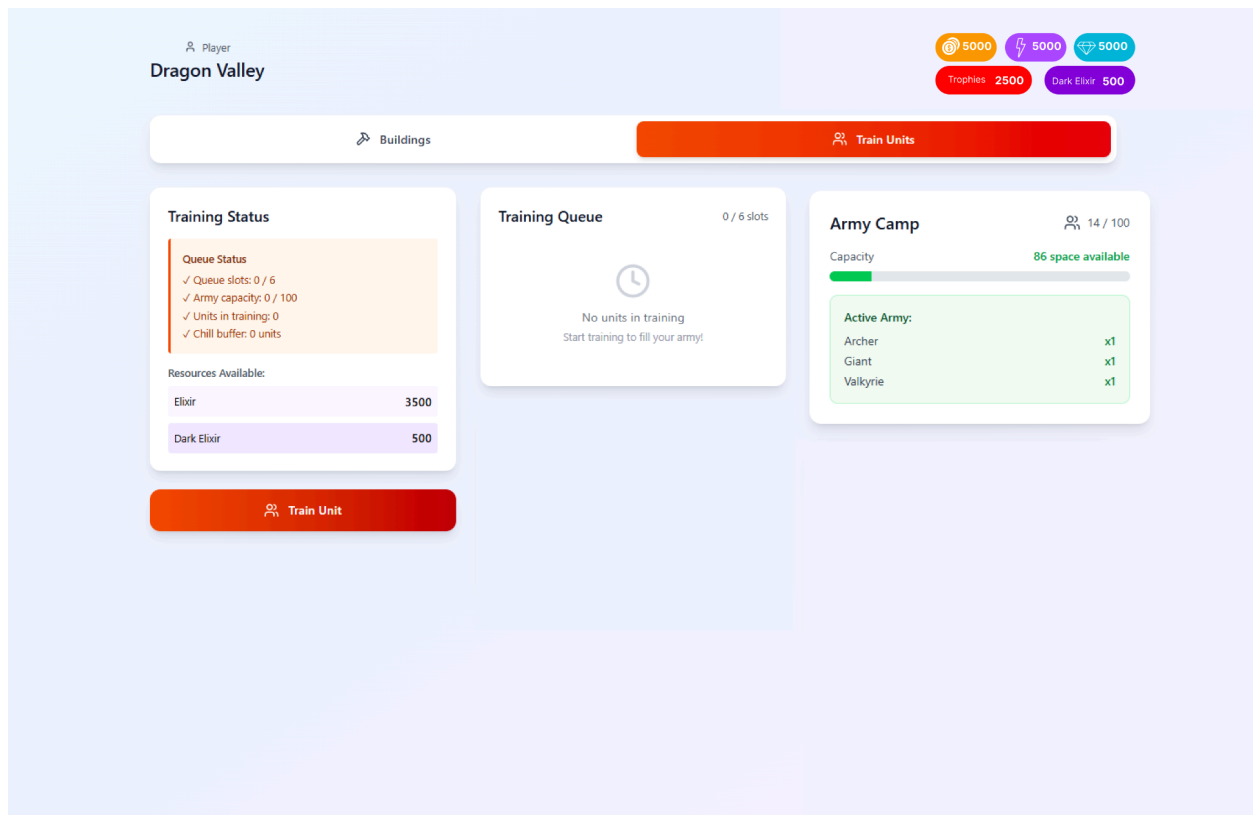
Allow the player to train units (troops or heroes), consume resources, and manage training queues. Players can choose on the above illustrated page between different types of units. If there are enough resources and current queue size is good with filling one more unit then it deducts the required resources.

Train Units training time in the queue (step 3)



The screen above shows the menu while training (we have the training queue with the units that player have chosen to train).

After the training - while units are moved to army camp (final stage)



This is the final state that occurs when the system transfers the unit to the army camp or when the cooling buffer is full (when capacity becomes available, the system will automatically transfer the unit from the cooling buffer to the army camp). In this case we see that the army camp still has 86 spaces available for upcoming units.

Alternative flow:

(step 2a) The system rejects the request and notifies the player of the missing resources - in a pop up message. No change with the unit appears - either no resources are deducted



Missing resources



Close

(step 2b) The system rejects the enqueue request and suggests clearing or expanding **the queue**. No change with the unit appears - either no resources are deducted

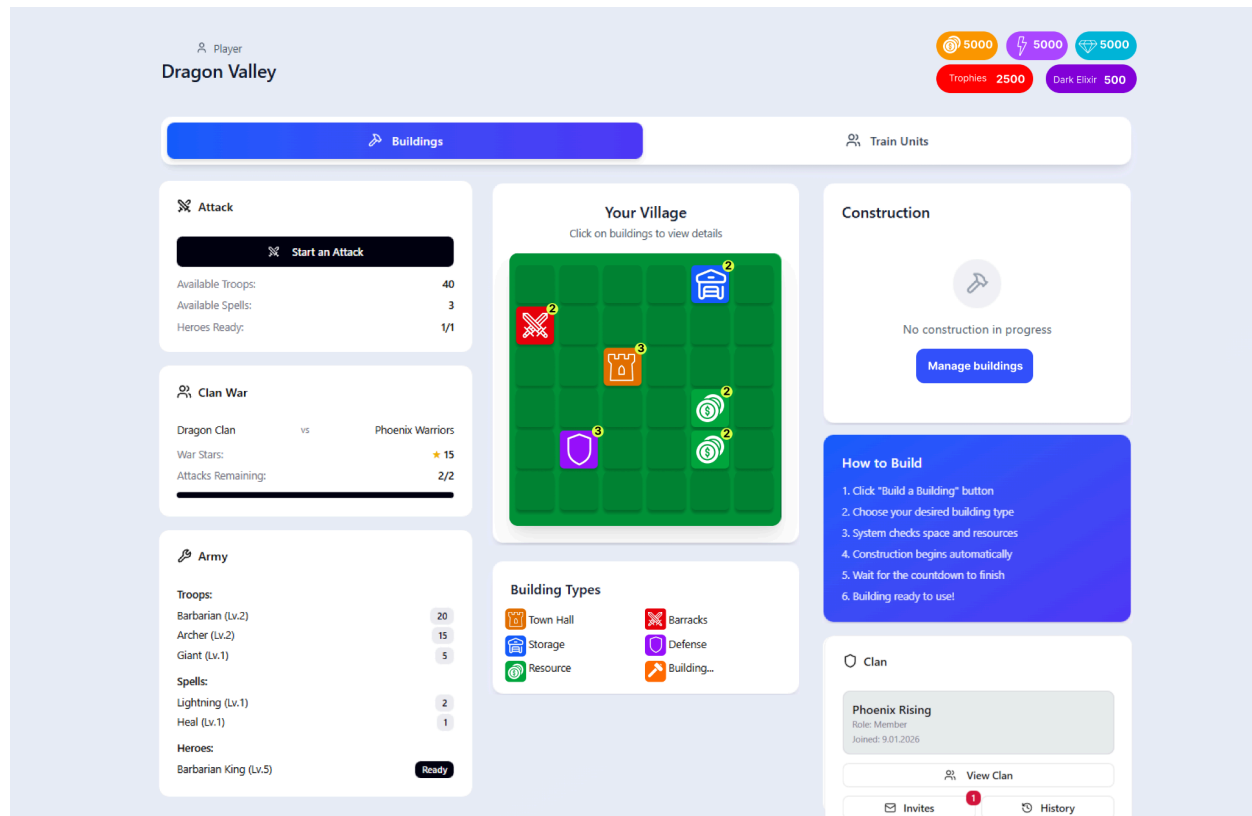


Please clear or expand the training queue



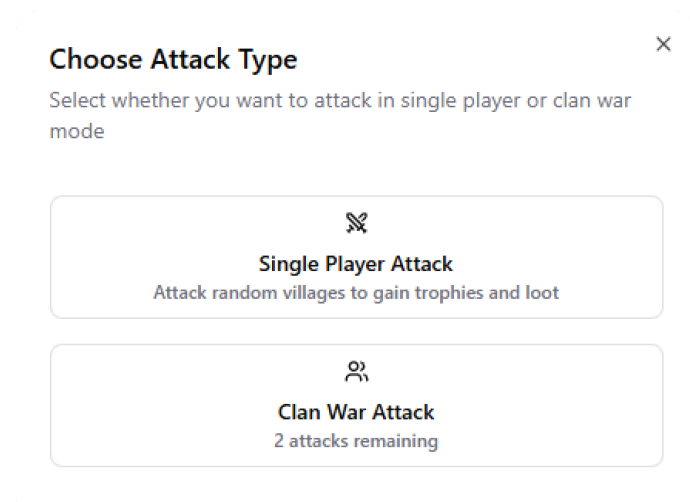
Close

Initiate an Attack and Apply Battle Result - initial state of the basic flow of events



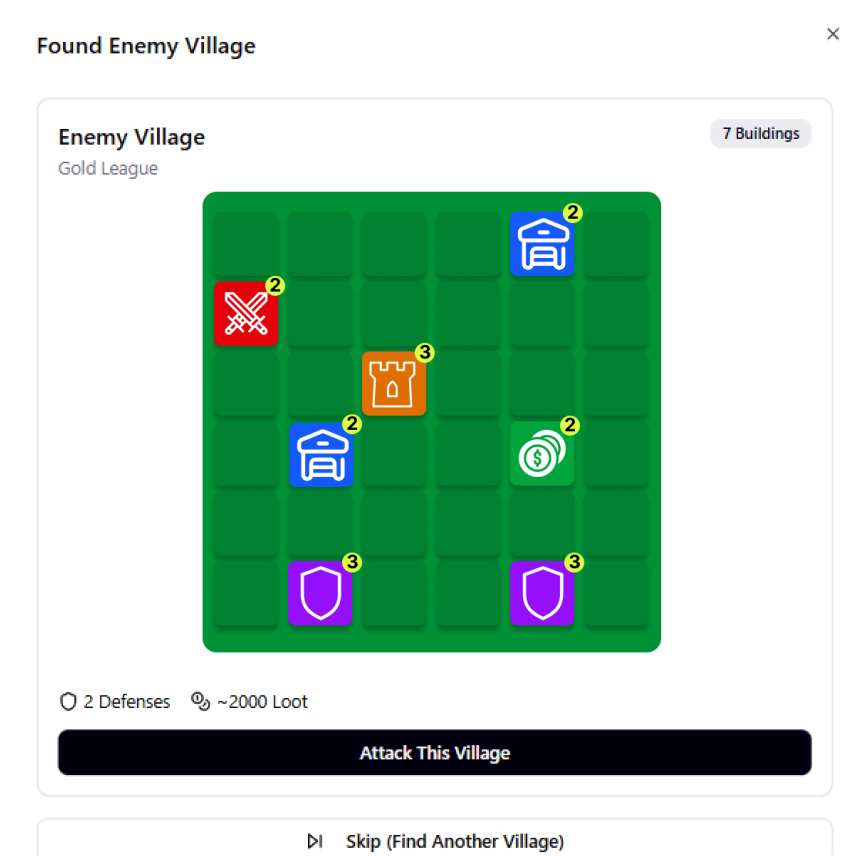
By clicking the "Start Attack" button, the system allows the player to launch various types of attacks – single attack and war (step 2). The system checks for resources for the attack and selects a random village from the same league as the attacker. After confirming the attack, the battle begins. The player can deploy their troops/spells/heroes on the map. The battle must end before the timer expires. After the system calculates the points (the player can win or lose), all trophies and resources earned are added up.

Choosing the type of attack (step 2)



The above screen shows the pop up page where the player can decide whether wants to do the single attack or clan war.

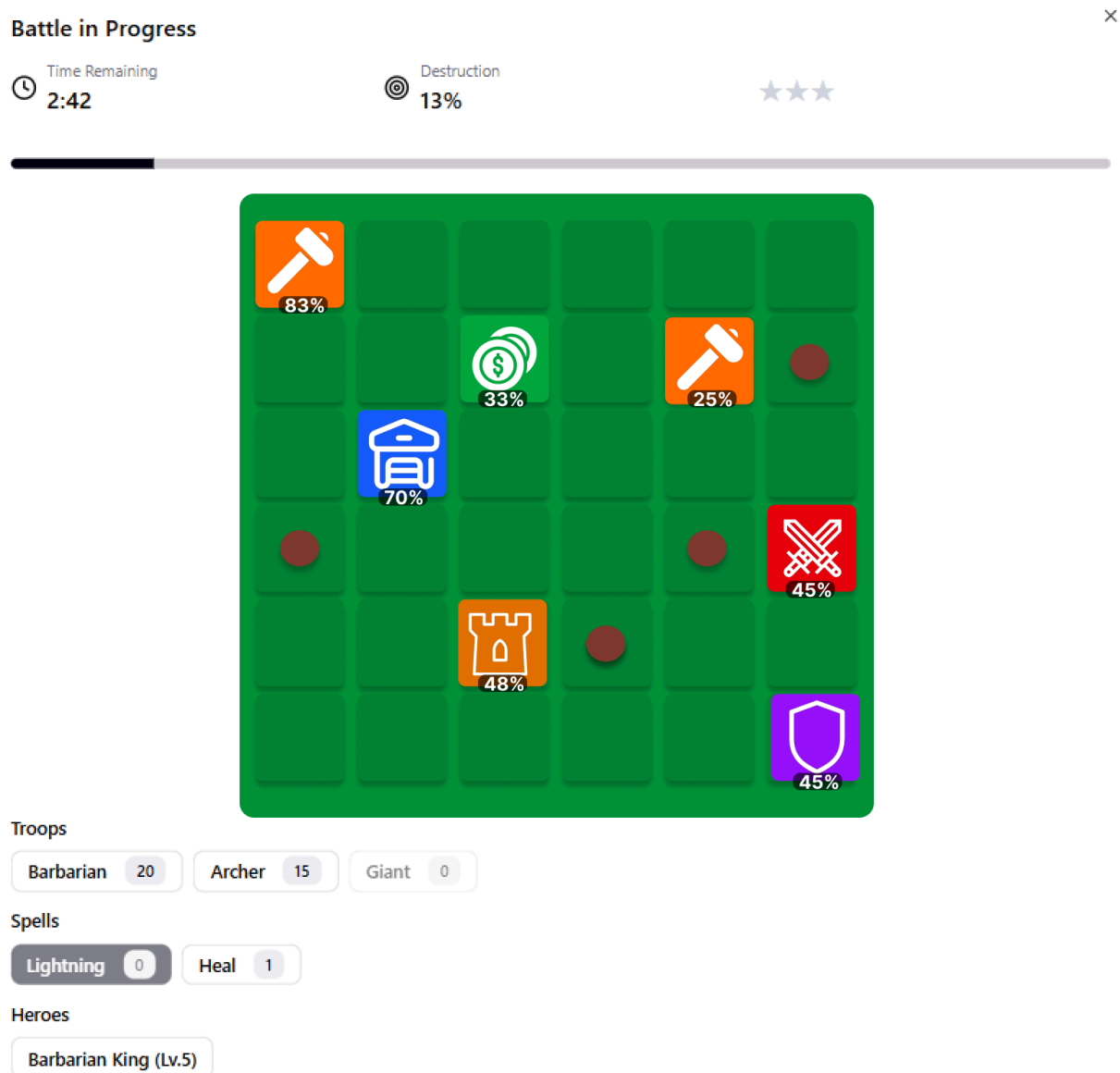
Choosing the random village from the same League for attack (step 4)



The above screen shows the pop up page where the player can decide to attack on the village or skip it (those two functionalities can do so by clicking on the appropriate button). On the screen also appears informations about the:

- loot gain
- How many buildings the village has
- Nick of the opponent
- League of the opponent
- The village with buildings (and their types along with levels)

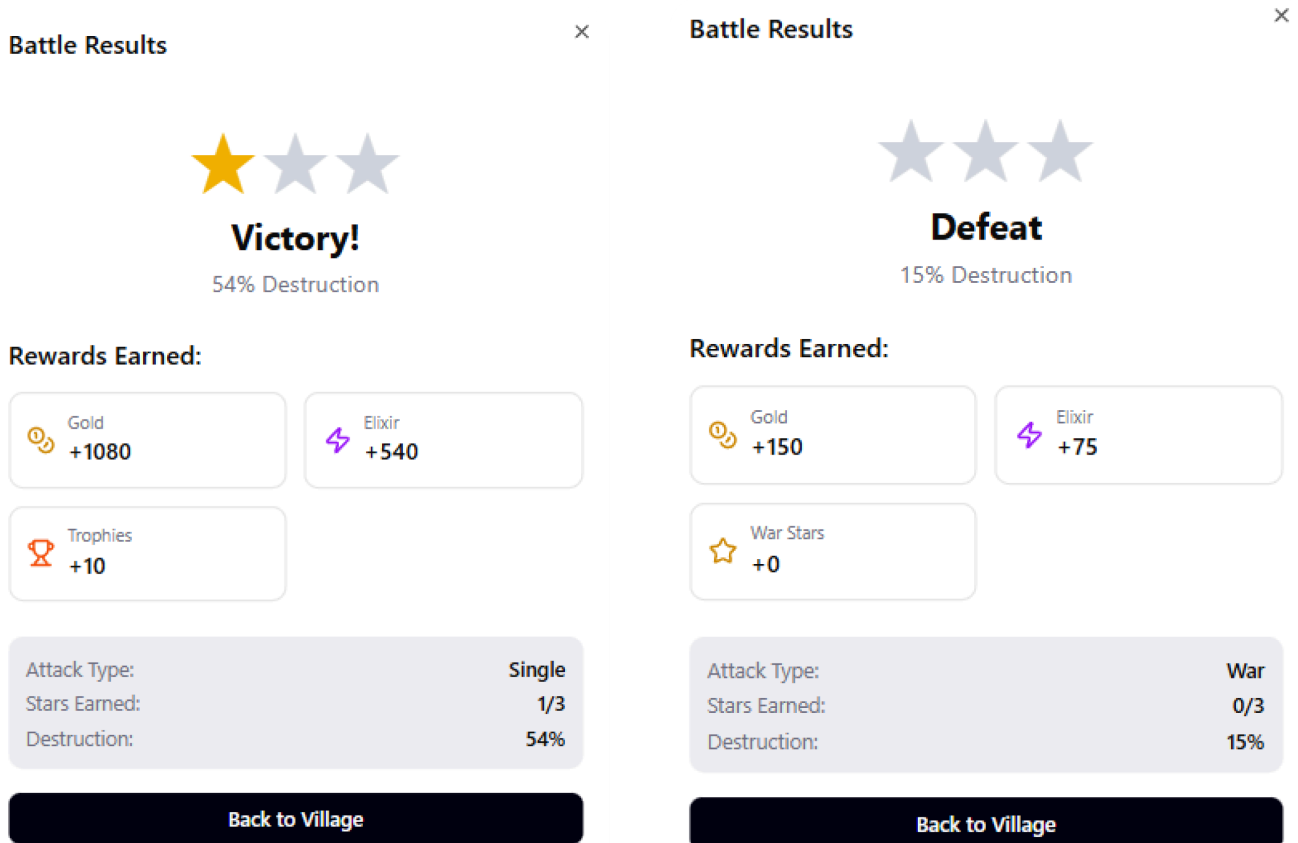
Battle time (step 7)



The screen above illustrates the main page where the battle/ attack takes place, player can place the Troops, Spells and Heroes on the map by clicking first one of mentioned above and

then the place on the map/ village of the opponent (the brown circles are the troops and heros) . The map consists of buildings (along with types of buildings shown as icons) that have its strength (expressed in %). The building will disappear from the map when it has 0% of strength. Player can also see on this page remaining time of the battle,destruction percentage and stars that will be awarded for winning.

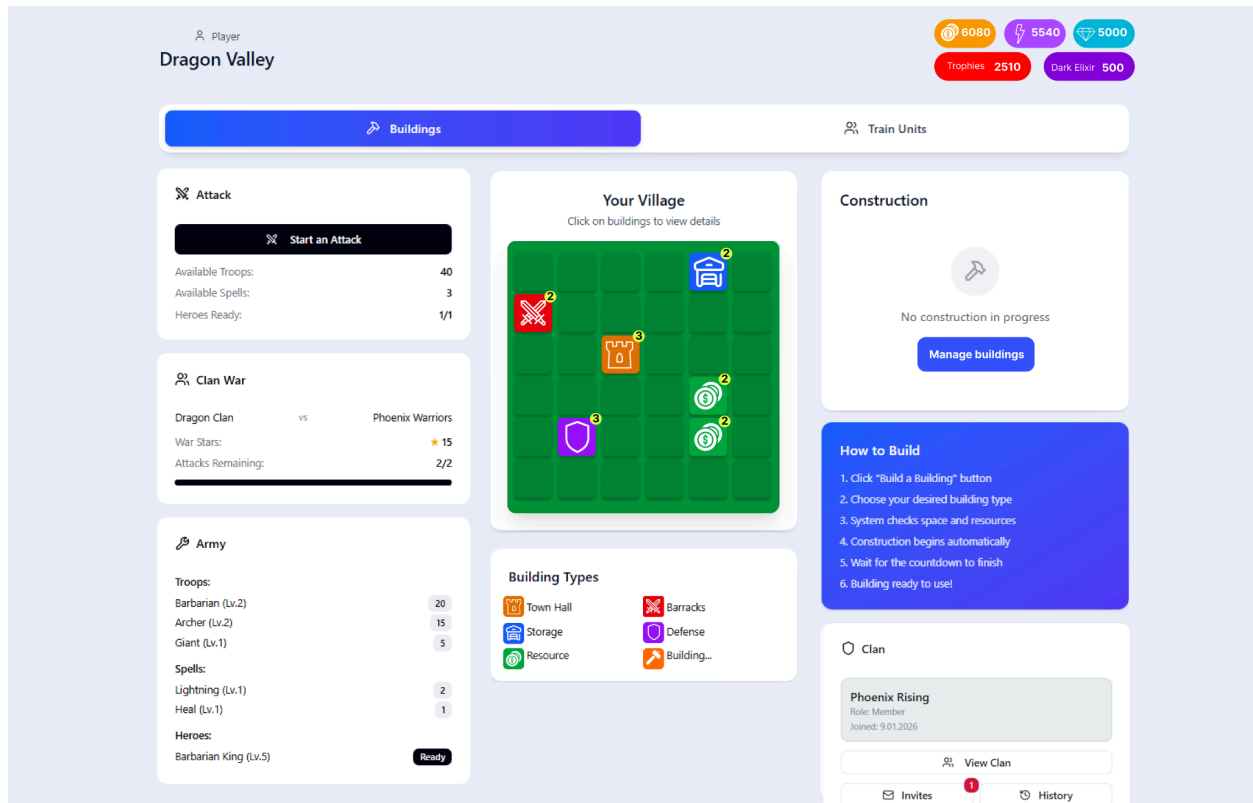
Victory and defeat page (step 13)



The shown above screens are two possible options after all of the attacks, Victory for winning and Defeat for losing. We have information on those with rewards earned during the attack and destruction percentage, also the player can see the stars for the battle.

The Player can come back to the village (main menu -building) by clicking the “back to village” or “x” button.

After the attack - single attack (final stage)



After the attack in the single player mode in the main menu not many things changed - besides the resources that have been updated according to the awards after the battle.

Alternative flow:

(Step 2a1) player chooses “war attack” - then if player has enough attempts then player chooses available village to attack (from the list of the opponent clan **Step -> 2a3**)

(Step 5a1) player skips showed village in a single attack (shown in the Step 4 -above), while clicking the button “Skip (find another one)” system will display next random village of the same league (**redirection to the step 4**)

(Step 2a2a1) Player doesn't have enough attempts. It will go back to the main menu building with no change in the system (**redirection to the main menu - building**)

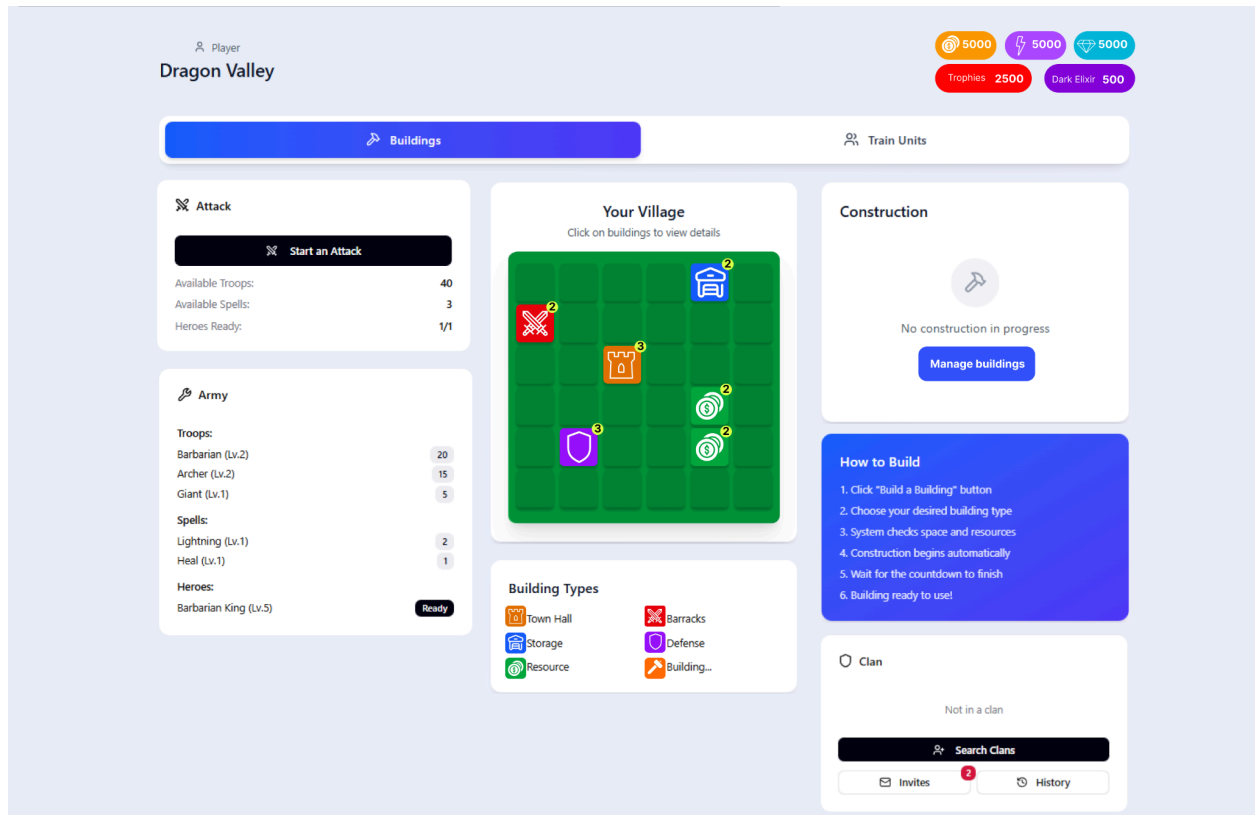
(Step 12a1) Applying results of a war attack. The system will reward players and the clan war with additional rewards like stars. (**redirection to the main menu - building**)

Choosing the village from the opponent clan (step 2a3)



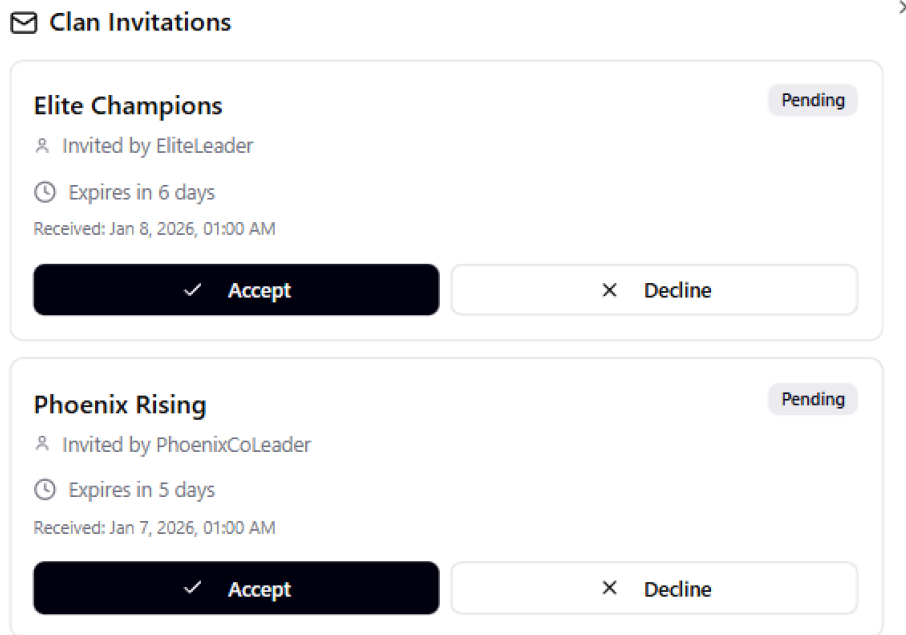
On the above screen we can see the pop up page where players can choose between different villages to attack in war attack mode. (The page is similar to step 4 with this difference that the player cannot skip to a different list of villages and have all of the available ones displayed in a form of list. To accept one to attack needs to click "Attack this village").

Join clan with ban- initial state of the basic flow of events



The above screen shows the main menu building with player not added to any clan. The Player can search for Clan, check invites and go through the history of the account. Then the system checks if the player is not banned from a given clan and creates membership.

Invitations from clans



The above screen illustrates the page where players can accept or decline invitations. Players can find information here when the invitation expires, by whom was sent and to which clan, also when received. By clicking Accept system will check if the player is not banned from the given clan

Successfully joined player to clan (step 5)



Successfully Joined!



Dragon Warriors



Welcome to the clan!

You can join this clan



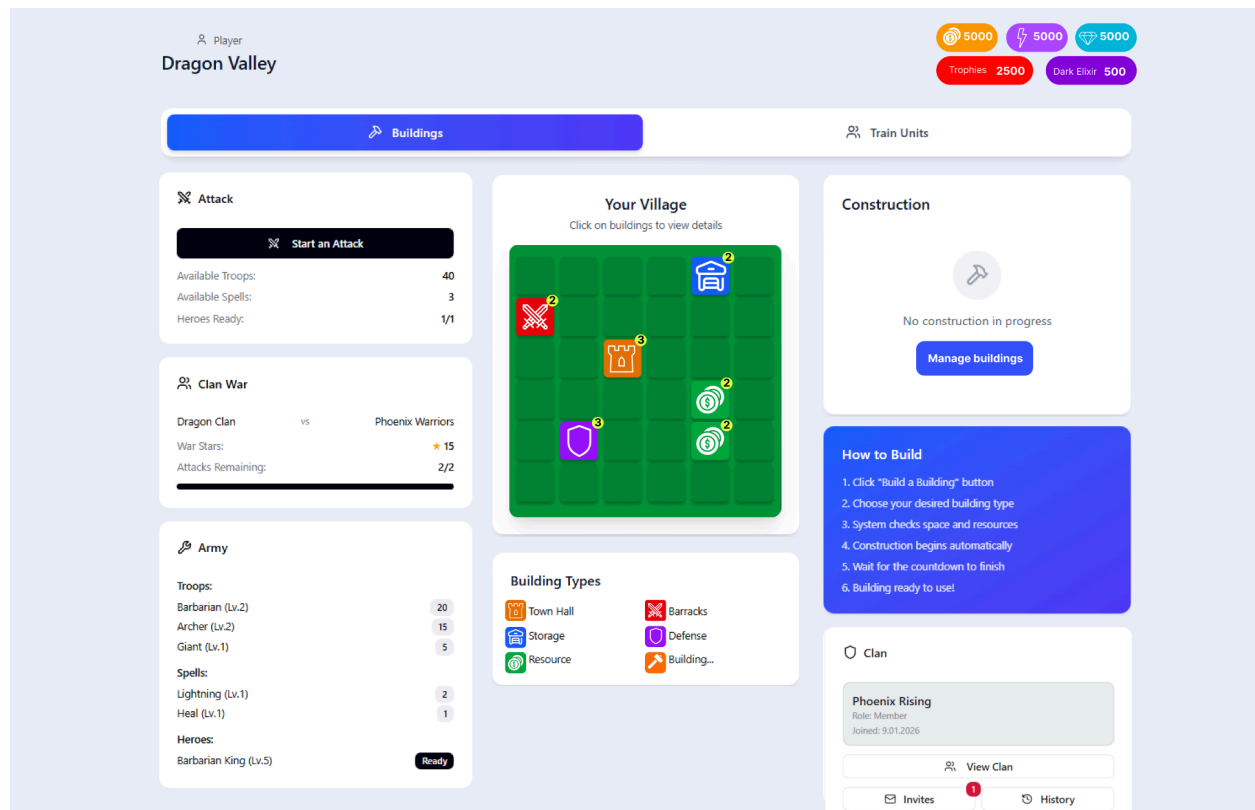
You are now a member of **Dragon Warriors!**

- Participate in clan wars
- Request and donate troops
- Chat with clan members
- Earn clan perks and rewards

Go to Clan

On the above screen player can find the information that was successfully added to the clan (the clan name) and also small info on what the member can do and the message that will be sent to the clan.

After the positive admission to the clan(final stage)



The above screen shows the main menu building with all its components plus now player can see information about the Clan (by clicking view clan and simple information in the box designed for functionalities of clan).

Still invites to new clans and history of the account is there and player can always see it by clicking on the button (functionalities explained above).

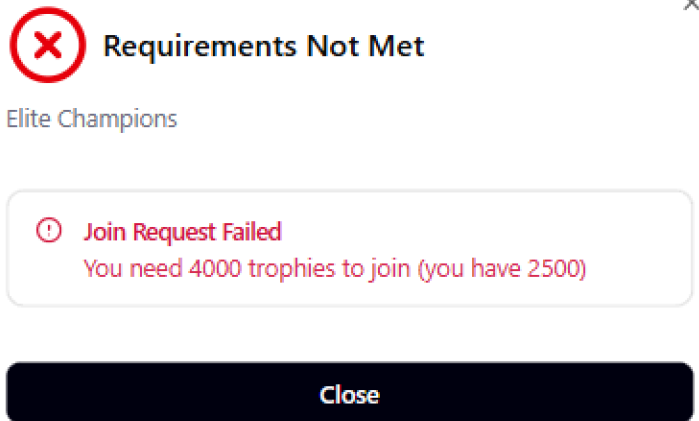
Alternatives:

(Step 3a) The Player is banned. The appropriate pop up page will be shown and the system will not add the player to the clan - no changes will appear in the main menu building.

(redirection to the main menu - building)

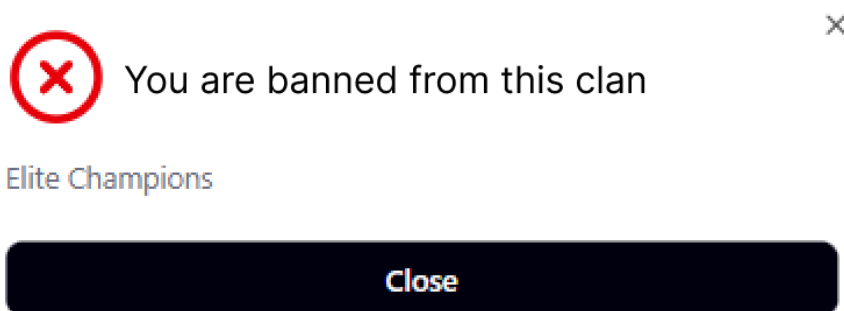
(Step 3b1) The Player is rejected by not enough capacity in the clan. The appropriate pop up page will be shown and the system will not add the player to the clan - no changes will appear in the main menu building. **(redirection to the main menu - building)**

Unsuccessfully joined to clan (Step 3b1)



The above screen shows the pop up page where the player can find information about why was rejected by the clan - in this case because the player doesn't have enough trophies. Player will come back to the main menu building by clicking close or x button

Unsuccessfully joined to clan (Step 3a)



The above screen shows the pop up page where the player can find information about why was rejected by the clan - in this case because the player was already banned from the given clan. Player will come back to the main menu building by clicking close or x button

Search Clans

Search Clans



Q Search by clan name or tag...



Dragon Warriors

Lv.15

Open

#DW2024

Elite clan for serious war players. Active donations required!

42/50

45 000

230 wins

Min: 2000

Join Clan



Phoenix Rising

Lv.10

Open

#PR2024

Friendly clan. All levels welcome. War twice per week.

35/50

32 000

156 wins

Min: 1000

Join Clan



Elite Champions

Lv.20

Invite Only

#EC2024

Invitation only. Top 100 war clan

The above screen shows the page where the player can find and join the clan (by clicking the “join clan” button). Players can find information like how many players currently are in the given clan, rewards, wins and minimum requirements to join. The level of the clan is shown as well.

History of the memberships in the clans

Membership History ✕

Active Bans (1)

Banned from Clan

Banned by Leader123 on Oct 15, 2025

Active

Reason: Toxic behavior

Expires: Apr 15, 2026

Past Memberships (2)

Old Warriors

 LEFT

Role: Member

 Joined: Nov 15, 2025

 Left: Dec 20, 2025

Banned Clan

 KICKED

Role: Member

 Joined: Oct 1, 2025

 Left: Oct 15, 2025

The above screen shows the pop up page where the player can see all information about its past activities in the clans, along with the information why he was banned from a given clan.